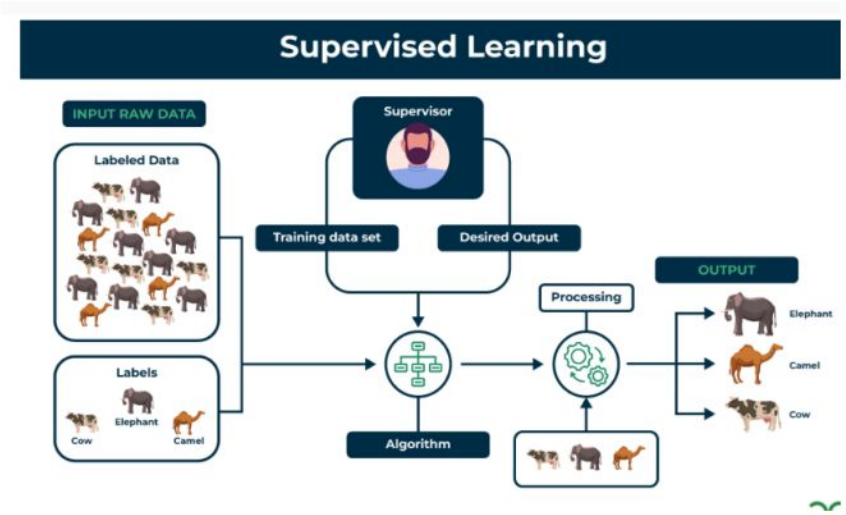
Supervised and Unsupervised learning

Supervised and unsupervised learning are two main types of machine learning. In supervised learning, the model is trained with labeled data where each input has a corresponding output. On the other hand, unsupervised learning involves training the model with unlabeled data which helps to uncover patterns, structures or relationships within the data without predefined outputs.

What is Supervised learning?

- Supervised learning as the name suggests, works like a **teacher** or **supervisor** guiding the machine.
- In this approach we teach or train the machine using the ***labelled data(***correct answers or classifications) which means each input has the correct output in the form of answer or category attached to it.
- After that machine is provided with a new set of examples (data) so that it can analyses the training data and produces a correct outcome from labeled data.
- **For example**, a labeled dataset of images of Elephant, Camel and Cow would have each image tagged with either "**Elephant**", "**Camel**" or "**Cow**."



Example to Understand

- Imagine we have a basket full of different fruits that we want the machine to identify.
- The machine first looks at the image of a fruit and extracts features like its shape, color and texture.
- Then it compares these features to the fruits it has already learned during training.

- ❖ If the new fruit's features closely match those of an apple, the machine will predict that the fruit is an apple.

For example, suppose we train the machine by showing it fruits one by one:

- If the fruit is round, has a small depression at the top and is red, it is labeled as an Apple.
- If the fruit is long, curved and greenish-yellow, it is labeled as a Banana.

Now after this training, if we give the machine a new fruit (say a banana) from the basket and ask it to identify it, the machine will use what it has learned during training. It will analyze the shape and color of the new fruit and classify it as a Banana placing it in the correct category. In this way, the machine learns from the training data (the basket with labeled fruits) and applies that knowledge to recognize new, unseen fruits.

Types of Supervised Learning

Supervised learning is classified into two types of algorithms:

1. Regression

A regression is used to predict **continuous** values such as house prices, stock prices or temperature. Regression algorithms learn how to connect input data to a specific number or value.

Some common regression algorithms include:

1. Linear Regression
2. Polynomial Regression
3. Lasso Regression
4. Ridge Regression

2. Classification

A classification is used to predict **categorical** values such as whether a customer will buy or not, whether an email is spam or not or whether a medical image shows a tumor or not. Classification algorithms learn how to connect input data to the probability of belonging to different groups or categories.

Some of the most common classification algorithms include:

1. Logistic Regression
2. Support Vector Machines
3. Decision Trees
4. Random Forests
5. Naive Baye

Applications of Supervised learning

It can be used to solve variety of problems which includes:

1. **Image classification:** It can automatically classify images into different categories such as animals, objects or scenes helps in the tasks like image search, content moderation and image-based product recommendations.
2. **Medical diagnosis:** It can assist in medical diagnosis by analyzing patient data such as medical images, test results and patient history to identify patterns that suggest specific diseases or conditions.
3. **Fraud detection:** They can analyze financial transactions and identify patterns that shows fraudulent activity which helps financial institutions prevent fraud and protect their customers.
4. **Natural language processing (NLP):** It plays a important role in NLP tasks including sentiment analysis, machine translation and text summarization which enables machines to understand and process human language effectively.

Advantages of Supervised learning

1. It learns from labeled examples to make accurate predictions on new, unseen data.
2. With more data and training, these models increases their accuracy which leads to better performance and more reliable predictions.
3. It works well for many tasks from detecting spam emails to predicting house prices as it has the ability to handle various computational challenges.
4. It can handle both classification (sorting data into categories) and regression (predicting numbers) which makes it flexible for different problems.

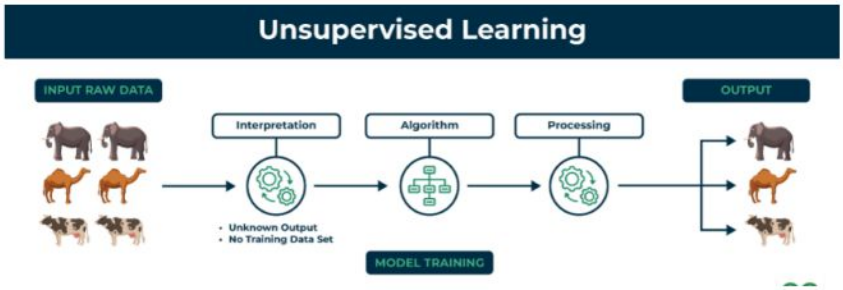
Disadvantages of Supervised learning

1. It requires a well-labeled dataset where each input has a corresponding output. Creating such datasets takes a lot of time, money and effort and can sometimes have mistakes, this makes supervised learning hard to use.
2. It works well on many tasks but can struggle with very complex or unstructured problems like understanding patterns or abstract ideas that doesn't relate to what it was trained on.
3. These models can sometimes overfit the training data which means they perform well on training data but poor on new, unseen data.
4. These models often need constant updating with new labeled data to stay accurate as real-world data changes over time.

What is Unsupervised learning?

- Unsupervised learning is a part of machine learning which works differently from supervised because there is no teacher(supervisor) involved to guide the machine.
- In this approach the machine is given with data that has **no labels or categories**. It analyzes the data on its own to find patterns, groups or relationships without any prior knowledge.
- The machine learns by discovering hidden structures within the data without being told what the correct output should be.

For example, unsupervised learning can analyze animal data and group the animals by their traits and behavior. These groups might represent different species which allows the machine to organize animals without any prior labels or categories.



Example to understand

- Imagine we have a machine learning model trained on many unlabeled images of dogs and cats.
- The model has never seen any labeled example that says “dog” or “cat” before so it doesn’t know how these animals look like.
- Now, if we give the model a new image that contains both dogs and cats it won’t be able to directly label them as “dog” or “cat.”
- It will group parts of the image based on similarities and differences in features like shape or texture.
- It might separate the image into two groups one with dog-like features and other with cat-like features.
- This happens because unsupervised learning doesn’t rely on prior knowledge or training with labeled data.
- It finds patterns and organizes data on its own helps in discovering information that wasn’t given before.

Types of Unsupervised Learning

Unsupervised learning is divided into two categories of algorithms:

1. Clustering

A clustering is used to group similar data points together. Clustering algorithms work by repeatedly moving data points closer to to the center of their group (cluster) and farther from points in other groups. This helps the algorithm to create clear and meaningful clusters. Some popular clustering algorithms include:

1. K-means clustering
2. Hierarchical clustering
3. Principal Component Analysis (PCA)
4. Singular Value Decomposition (SVD)
5. Independent Component Analysis
6. Gaussian Mixture Models (GMMs)
7. Density-Based Spatial Clustering of Applications with Noise (DBSCAN)

2. Association rule learning

An association rule learning used to find patterns and relationships between different items in a dataset. It looks for rules like "people who buy X often also buy Y".

Some common Association rule learning algorithms include:

1. Apriori Algorithm
2. Eclat Algorithm
3. FP-Growth Algorithm

Application of Unsupervised learning

Unsupervised learning can be used to solve a variety of problems which includes:

1. **Anomaly detection**: It can identify unusual patterns or behaviors in data helps in the detection of fraud, security breaches or system problems.
2. **Scientific discovery**: It can show hidden relationships and patterns in scientific data which leads to new insights and ideas.
3. **Recommendation systems**: It finds similarities in user behavior and preferences to recommend products, movies or music that align with their interests.
4. **Customer segmentation**: It can identify groups of customers with similar characteristics which allows businesses to target marketing campaigns and improve customer service more effectively.

Advantages of Unsupervised learning

1. It doesn't need labeled data so we can start working with large datasets more easily and quickly.
2. This handles large amounts of data and reduce it into simpler forms without losing important patterns which makes it manageable and efficient.
3. It discovers patterns and relationships in the data that were previously unknown which offers valuable insights.
4. By analyzing unlabeled data, it shows meaningful trends and groups that help us to understand our data deeply.

Disadvantages of Unsupervised learning

1. Without labeled answers, it's difficult to tell how accurate or effective the model is.
2. Lack of clear guidance can lead to less precise results for complex problems.
3. After grouping the data, we may needs to check and label these groupings which can be time-consuming.
4. Missing data, outliers or noise in the data can easily affect the quality of the results.

Underfitting and Overfitting

Machine learning models aim **to perform well on both training data and new, unseen data and is considered "good"** if:

1. It ***learns patterns effectively from the training data***.
2. It ***generalizes well to new, unseen data***.
3. It ***avoids memorizing the training data (overfitting) or failing to capture relevant patterns (underfitting).***

To evaluate how well a model learns and generalizes, we monitor its performance on both the training data and a separate validation or test dataset which is often measured by its **accuracy** or **prediction errors**. However, achieving this balance **can be challenging**. ***Two common issues that affect a model's performance and generalization ability are*** *overfitting* ***and*** *underfitting*. These problems are major contributors to poor performance in machine learning models. Let's us understand what they are and how they contribute to ML models.

Bias and Variance in Machine Learning

Bias and **variance** are two key sources of error in machine learning models that directly impact their performance and generalization ability.

Bias: ***is the error that happens when a machine learning model is too simple and doesn't learn enough details from the data. It's like assuming all birds can only be small and fly, so the model fails to recognize big birds like ostriches or penguins that can't fly and get biased with predictions.***

- These assumptions make the model easier to train but may prevent it from capturing the underlying complexities of the data.
- ***High bias typically leads to** underfitting**, where the model performs poorly on both training and testing data because it fails to learn enough from the data.***
- **Example**: **A linear regression model applied to a dataset with a non-linear relationship.**

Variance: ***Error that happens when a machine learning model learns too much from the data, including random noise.***

- A high-variance model learns not only the patterns but also the noise in the training data, which leads to poor generalization on unseen data.
- ***High variance typically leads to** overfitting**, where the model performs well on training data but poorly on testing data.***

Overfitting and Underfitting: The Core Issues

1. Overfitting in Machine Learning

Overfitting happens when a model learns too much from the training data, including details that don't matter (like noise or outliers).

- For example, **imagine fitting a very complicated curve to a set of points. The curve will go through every point, but it won't represent the actual pattern.**
- As a result, the **model works great on training data but fails when tested on new data**.

Overfitting models are like students who memorize answers instead of understanding the topic. They do well in practice tests (training) but struggle in real exams (testing).

Reasons for Overfitting:

1. High variance and low bias.
2. The model is too complex.
3. The size of the training data.

2. Underfitting in Machine Learning

Underfitting is the opposite of overfitting. It happens **when a model is too simple to capture what's going on in the data.**

- For example, **imagine drawing a straight line to fit points that actually follow a curve. The line misses most of the pattern.**
- In this case, *the model doesn't work well on either the training or testing data.*

Underfitting models are like students who don't study enough. They don't do well in practice tests or real exams.

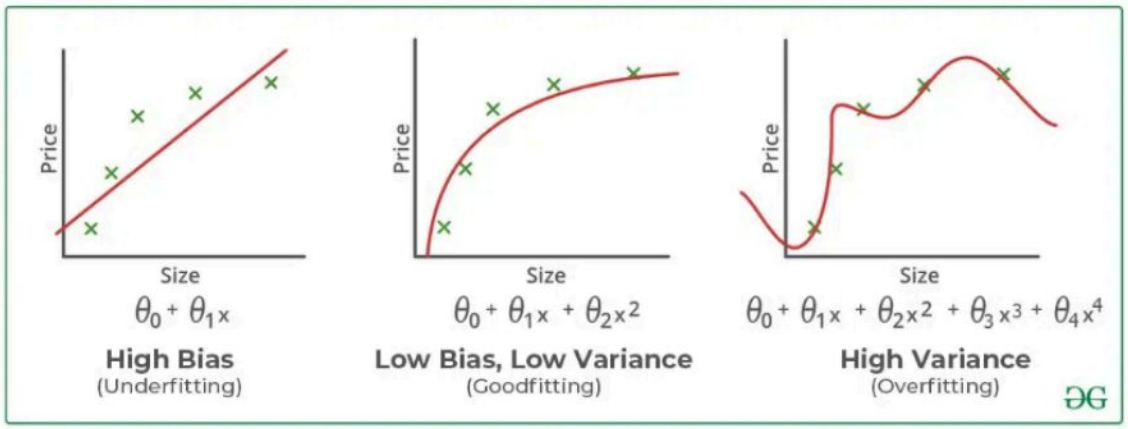
Note: The underfitting model has High bias and low variance.

Reasons for Underfitting:

1. The model is too simple, So it may be not capable to represent the complexities in the data.
2. The input features which is used to train the model is not the adequate representations of underlying factors influencing the target variable.
3. The size of the training dataset used is not enough.

4. Excessive regularization are used to prevent the overfitting, which constraint the model to capture the data well.
5. Features are not scaled.

Let's visually understand the concept of **underfitting, proper fitting, and overfitting**.



- **Underfitting : S**traight line trying to fit a curved dataset but cannot capture the data's patterns, leading to poor performance on both training and test sets.
- **Overfitting**: A squiggly curve passing through all training points, failing to generalize performing well on training data but poorly on test data.
- **Appropriate Fitting**: Curve that follows the data trend without overcomplicating to capture the true patterns in the data.

Balance Between Bias and Variance

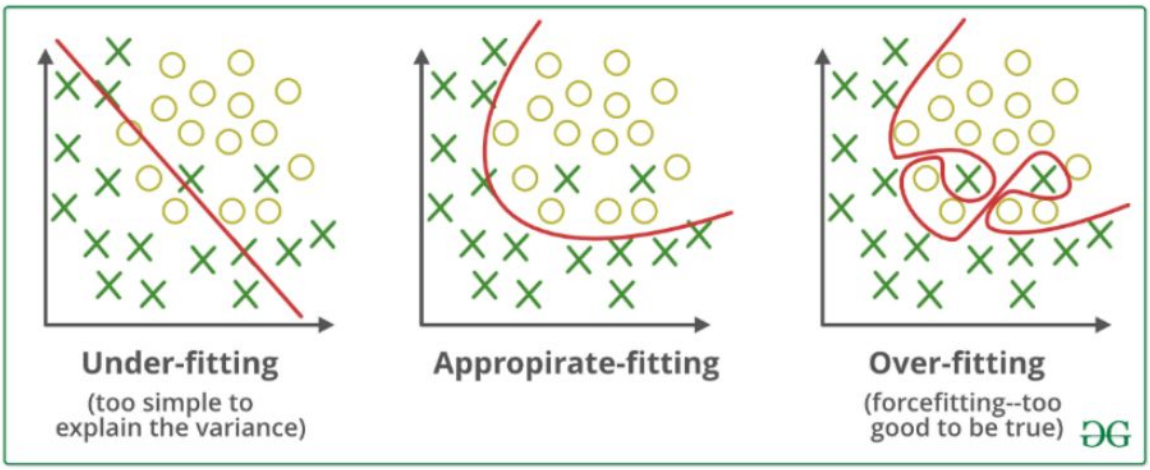
The relationship between bias and variance is often referred to as the bias-variance tradeoff**,** which highlights the need for balance:

- Increasing model complexity reduces bias but increases variance (risk of overfitting).

- Simplifying the model reduces variance but increases bias (risk of underfitting).

The goal is to find an optimal balance where both bias and variance are minimized, resulting in good generalization performance.

Imagine you're trying to predict the price of houses based on their size, and you decide to draw a line or curve that best fits the data points on a graph. How well this line captures the trend in the data depends on the complexity of the model you use.



Underfitting and Overfitting

- When a model is too simple, like fitting a straight line to curved data, it has **high bias** and fails to capture the true relationship, leading to **underfitting**. For example, a linear model cannot represent a non-linear increase in house prices with size.
- However, if the model becomes too complex, like a fourth-degree polynomial that adjusts to every point, it develops **high variance**, overfits the training data, and struggles to generalize to new data. This is **overfitting**, where the model performs well on training but poorly on testing.

- An ideal model strikes a balance with **low bias and low variance**, capturing the overall pattern without overreacting to noise. For instance, a smooth second-degree polynomial fits the data well without being overly complex.

How to Address Overfitting and Underfitting?

Techniques to Reduce Underfitting

1. Increase model complexity.
2. Increase the number of features, performing feature engineering.
3. Remove noise from the data.
4. Increase the number of epochs or increase the duration of training to get better results.

Techniques to Reduce Overfitting

1. Improving the quality of training data reduces overfitting by focusing on meaningful patterns, mitigate the risk of fitting the noise or irrelevant features.
2. Increase the training data can improve the model's ability to generalize to unseen data and reduce the likelihood of overfitting.
3. Reduce model complexity.
4. Early stopping during the training phase (have an eye over the loss over the training period as soon as loss begins to increase stop training).
5. Ridge Regularization and Lasso Regularization.
6. Use dropout for neural networks to tackle overfitting.

Linear Regression in Machine learning

Linear regression is a type of **supervised machine-learning algorithm** that learns from the **labelled datasets** and maps the data points with most **optimized linear functions which can be used for prediction on new datasets**. It assumes that there is a **linear relationship between the input and output, meaning the output changes at a constant rate as the input changes.** This relationship is represented by a straight line.

For example we want to predict a student's exam score based on how many hours they studied. We observe that as students study more hours, their scores go up. In the example of predicting exam scores based on hours studied. Here

- **Independent variable (input):** Hours studied because it's the factor we control or observe.
- **Dependent variable (output):** Exam score because it depends on how many hours were studied.

We use the independent variable to predict the dependent variable

Why Linear Regression is Important?

Here's why linear regression is important:

- **Simplicity and Interpretability:** It's easy to understand and interpret, making it a starting point for learning about machine learning.
- **Predictive Ability**: Helps predict future outcomes based on past data, making it useful in various fields like finance, healthcare and marketing.
- **Basis for Other Models:** Many advanced algorithms, like logistic regression or neural networks, build on the concepts of linear regression.
- **Efficiency:** It's computationally efficient and works well for problems with a linear relationship.

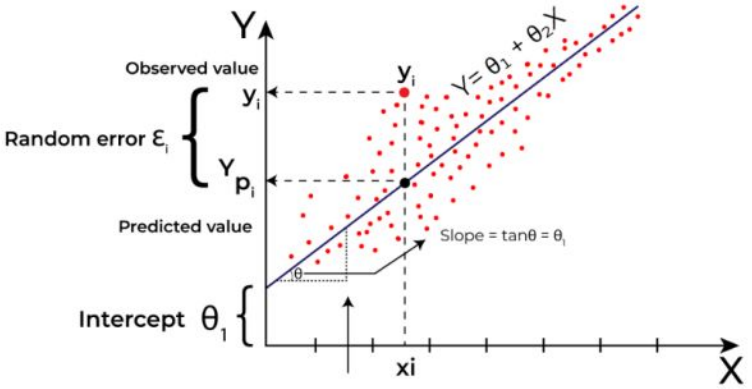
- **Widely Used:** It's one of the most widely used techniques in both statistics and machine learning for regression tasks.
- **Analysis:** It **provides insights into relationships between variables** (e.g., how much one variable influences another).

Best Fit Line in Linear Regression

In linear regression, the **best-fit line is the straight line** that most accurately represents the relationship between the independent variable (input) and the dependent variable (output). It is the line that minimizes the difference between the actual data points and the predicted values from the model.

1. Goal of the Best-Fit Line

The goal of linear regression **is to find a straight line that minimizes the error (the difference) between the observed data points and the predicted values.** This line helps us predict the dependent variable for new, unseen data.



Linear Regression

Here Y is called a dependent or target variable and X is called an independent variable also known as the predictor of Y. There are many types of functions or modules that can be used for regression. A linear function is the simplest type of function. Here, X may be a single feature or multiple features representing the problem.

2. Equation of the Best-Fit Line

For simple linear regression (with one independent variable), the best-fit line is represented by the equation

$$y=mx+b$$

Where:

- **y** is the predicted value (dependent variable)
- **x** is the input (independent variable)
- **m** is the slope of the line (how much y changes when x changes)
- **b** is the intercept (the value of y when x = 0)

The best-fit line will be the one that optimizes the values of m (slope) and b (intercept) so that the predicted y values are as close as possible to the actual data points.

3. Minimizing the Error: The Least Squares Method

To find the best-fit line, we use a method called **Least Squares**. The idea behind this method is to minimize the sum of squared differences between the actual values (data points) and the predicted values from the line. These differences are called residuals.

The formula for residuals is:

$$Residual=y_i-\hat{y}_i$$

Where:

- y_i is the actual observed value
- $\hat{y}_i$ is the predicted value from the line for that x_i

The least squares method minimizes the sum of the squared residuals:

$$Sum\ of\ squared\ errors(SSE)=\Sigma(y_i-\hat{y}_i)^2$$

This method ensures that the line best represents the data where the sum of the squared differences between the predicted values and actual values is as small as possible.

4. Interpretation of the Best-Fit Line

- **Slope (m):** The slope of the best-fit line indicates how much the dependent variable (y) changes with each unit change in the independent variable (x). For example if the slope is 5, it means that for every 1-unit increase in x, the value of y increases by 5 units.
- **Intercept (b):** The intercept represents the predicted value of y when x = 0. It's the point where the line crosses the y-axis.

In linear regression some hypothesis are made to ensure reliability of the model's results.

Limitations

- ***Assumes Linearity:*** *The method assumes the relationship between the variables is linear.* ***If the relationship is non-linear, linear regression might not work well.***
- ***Sensitivity to Outliers:*** ***Outliers can significantly affect the slope and intercept, skewing the best-fit line.***

Hypothesis function in Linear Regression

In linear regression, the hypothesis function is the equation used to make predictions about the dependent variable based on the independent variables. It represents the relationship between the input features and the target output.

For a simple case with one independent variable, the hypothesis function is:

$$h(x)=\beta_0+\beta_1 x$$

Where:

- h(x)(or y^) is the predicted value of the dependent variable (y).
- x x is the independent variable.
- β_0 is the intercept, representing the value of y when x is 0.
- β_1 is the slope, indicating how much y changes for each unit change in x.

For **multiple linear regression** (with more than one independent variable), the hypothesis function expands to:

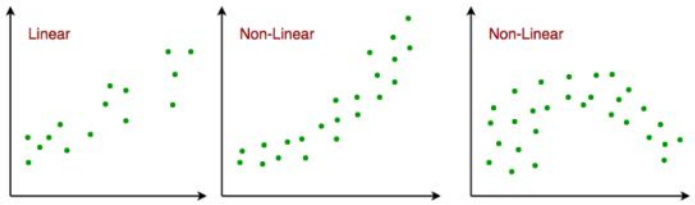
$$h(x_1,x_2,...,x_n)= \beta_0+ \beta_1x_1+ \beta_2x_2+...+ \beta_nx_n$$

Where:

- $x_1,x_2,...,x_n$ are the independent variables.
- β_0 is the intercept.
- $\beta_1, \beta_2,..., \beta_n$ are the coefficients, representing the influence of each respective independent variable on the predicted output.

Assumptions of the Linear Regression

1. **Linearity**: The relationship between inputs (X) and the output (Y) is a straight line.



2. *Linearity*

2. Independence of Errors: The errors in predictions should not affect each other.

3. Constant Variance (Homoscedasticity): The errors should have equal spread across all values of the input. If the spread changes (like fans out or shrinks), it's called heteroscedasticity and it's a problem for the model.

4. Normality of Errors: The errors should follow a normal (bell-shaped) distribution.

5. No Multicollinearity(for multiple regression): Input variables shouldn't be too closely related to each other.

6. No Autocorrelation: Errors shouldn't show repeating patterns, especially in time-based data.

7. Additivity: The total effect on Y is just the sum of effects from each X, no mixing or interaction between them.

1. Simple Linear Regression

Simple linear regression is used when we want to predict a target value (dependent variable) using only one input feature (independent variable). It assumes a straight-line relationship between the two.

Formula:

$$y^=\theta 0+\theta 1x$$

Where:

- y^ is the predicted value
- x is the input (independent variable)
- θ0 is the intercept (value of y^ when x=0)
- θ1 is the slope or coefficient (how much y^ changes with one unit of x)

Example:

Predicting a person's salary (y) based on their years of experience (x).

2. Multiple Linear Regression

Multiple linear regression involves more than one independent variable and one dependent variable. The equation for multiple linear regression is:

$$y^=\theta 0+\theta 1x1+\theta 2x2+\cdots+\theta nxn$$

where:

- y^ is the predicted value
- x1,x2,…,xn are the independent variables
- θ1,θ2,…,θn are the coefficients (weights) corresponding to each predictor.
- θ0 is the intercept.

The goal of the algorithm is to find the best Fit Line equation that can predict the values based on the independent variables.

In regression set of records are present with X and Y values and these values are used to learn a function so if you want to predict Y from an unknown X this learned function can be used. In regression we have to find the value of Y, So, a function is required that predicts continuous Y in the case of regression given X as independent features.

Use Case of Multiple Linear Regression

Multiple linear regression allows us to analyze relationship between multiple independent variables and a single dependent variable.

Here are some use cases:

- **Real Estate Pricing:** In real estate MLR is used to predict property prices based on multiple factors such as location, size, number of bedrooms, etc. This helps buyers and sellers understand market trends and set competitive prices.
- **Financial Forecasting:** Financial analysts use MLR to predict stock prices or economic indicators based on multiple influencing factors such as interest rates, inflation rates and market trends. This enables better investment strategies and risk management24.
- **Agricultural Yield Prediction:** Farmers can use MLR to estimate crop yields based on several variables like rainfall, temperature, soil quality and fertilizer usage. This information helps in planning agricultural practices for optimal productivity

- **E-commerce Sales Analysis:** An e-commerce company can utilize MLR to assess how various factors such as product price, marketing promotions and seasonal trends impact sales.

Now that we have understood about linear regression, its assumption and its type now we will learn how to make a linear regression model.

Cost function for Linear Regression

As we have discussed earlier about best fit line in linear regression, its not easy to get it easily in real life cases so we need to calculate errors that affects it. These errors need to be calculated to mitigate them. The difference between the predicted value Y^ and the true value Y and it is called cost function or the loss function.

In Linear Regression, the Mean Squared Error (MSE) cost function is employed, which calculates the average of the squared errors between the predicted values y^i and the actual values yi. The purpose is to determine the optimal values for the intercept θ1 and the coefficient of the input feature θ2 providing the best-fit line for the given data points. The linear equation expressing this relationship is y^i=θ1+θ2xi.

MSE function can be calculated as:

$$\text{Cost function}(J)=n1\sum ni(yi\hat{}-yi)2$$

Utilizing the MSE function, the iterative process of gradient descent is applied to update the values of \θ1&θ2. This ensures that the MSE value converges to the global minima, signifying the most accurate fit of the linear regression line to the dataset.

This process involves continuously adjusting the parameters \(\theta_1\) and \(\theta_2\) based on the gradients calculated from the MSE. The final result is a linear regression line that minimizes the overall squared differences between the

predicted and actual values, providing an optimal representation of the underlying relationship in the data.

Now we have calculated loss function we need to optimize model to mtigate this error and it is done through gradient descent.

Gradient Descent for Linear Regression

Gradient descent is an optimization technique used to train a linear regression model by minimizing the prediction error. It works by starting with random model parameters and repeatedly adjusting them to reduce the difference between predicted and actual values.



Gradient Descent

How it works:

- Start with random values for slope and intercept.
- Calculate the error between predicted and actual values.
- Find how much each parameter contributes to the error (gradient).
- Update the parameters in the direction that reduces the error.
- Repeat until the error is as small as possible.

This helps the model find the best-fit line for the data.

For more details you can refer to: Gradient Descent in Linear Regression

Evaluation Metrics for Linear Regression

A variety of evaluation measures can be used to determine the strength of any linear regression model. These assessment metrics often give an indication of how well the model is producing the observed outputs.

The most common measurements are:

1. Mean Square Error (MSE)

Mean Squared Error (MSE) is an evaluation metric that calculates the average of the squared differences between the actual and predicted values for all the data points. The difference is squared to ensure that negative and positive differences don't cancel each other out.

$$MSE = 1n \sum i = 1n (y_i - yi^{\wedge})2$$

Here,

- n is the number of data points.
- Yi is the actual or observed value for theith*ith*data point.
- yi^ is the predicted value for the ith data point.

MSE is a way to quantify the accuracy of a model's predictions. MSE is sensitive to outliers as large errors contribute significantly to the overall score.

2. Mean Absolute Error (MAE)

Mean Absolute Error is an evaluation metric used to calculate the accuracy of a regression model. MAE measures the average absolute difference between the predicted values and actual values.

Mathematically MAE is expressed as:

$$MAE = 1n \sum i = 1n |Yi - Yi^{\wedge}|$$

Here,

- n is the number of observations
- Yi represents the actual values.
- Yi^ represents the predicted values

Lower MAE value indicates better model performance. It is not sensitive to the outliers as we consider absolute differences.

3. Root Mean Squared Error (RMSE)

The square root of the residuals' variance is the Root Mean Squared Error. It describes how well the observed data points match the expected values or the model's absolute fit to the data.

In mathematical notation, it can be expressed as:

$$RMSE = \sqrt{\frac{RSS}{n}} = \sqrt{\frac{\sum_{i=2}^{n}(y_i^{actual} - y_i^{predicted})^2}{n}}$$

Rather than dividing the entire number of data points in the model by the number of degrees of freedom, one must divide the sum of the squared residuals to obtain an unbiased estimate. Then, this figure is referred to as the Residual Standard Error (RSE).

In mathematical notation, it can be expressed as:

$$RMSE = \sqrt{\frac{RSS}{n}} = \sqrt{\frac{\sum_{i=2}^{n}(y_i^{actual} - y_i^{predicted})^2}{(n-2)}}$$

RSME is not as good of a metric as R-squared. Root Mean Squared Error can fluctuate when the units of the variables vary since its value is dependent on the variables' units (it is not a normalized measure).

4. Coefficient of Determination (R-squared)

R-Squared is a statistic that indicates how much variation the developed model can explain or capture. It is always in the range of 0 to 1. In general, the better

the model matches the data, the greater the R-squared number.

In mathematical notation, it can be expressed as:

$$R^2 = 1 - \left(\frac{RSS}{TSS}\right)$$

- **Residual sum of Squares**(RSS): The sum of squares of the residual for each data point in the plot or data is known as the residual sum of squares or RSS. It is a measurement of the difference between the output that was observed and what was anticipated.

$$RSS = \sum_{i=1}^{n}(y_i - b_0 - b_1 x_i)^2$$

- **Total Sum of Squares (TSS):** The sum of the data points' errors from the answer variable's mean is known as the total sum of squares or TSS.

$$TSS = \sum_{i=1}^{n}(y - \overline{y_i})^2.$$

R squared metric is a measure of the proportion of variance in the dependent variable that is explained the independent variables in the model.

5. Adjusted R-Squared Error

Adjusted R2measures the proportion of variance in the dependent variable that is explained by independent variables in a regression model. Adjusted R-square accounts the number of predictors in the model and penalizes the model for including irrelevant predictors that don't contribute significantly to explain the variance in the dependent variables.

Mathematically, adjusted R^2 is expressed as:

$$Adjusted\, R^2 = 1 - \left(\frac{(1-R^2).(n-1)}{n-k-1}\right)$$

Here,

- n is the number of observations

- k is the number of predictors in the model
- R^2 is coeeficient of determination

Adjusted R-square helps to prevent overfitting. It penalizes the model with additional predictors that do not contribute significantly to explain the variance in the dependent variable.

While evaluation metrics help us measure the performance of a model, regularization helps in improving that performance by addressing overfitting and enhancing generalization.

Regularization Techniques for Linear Models

1. Lasso Regression (L1 Regularization)

Lasso Regression is a technique used for regularizing a linear regression model, it adds a penalty term to the linear regression objective function to prevent overfitting.

The objective function after applying lasso regression is:

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^{m} (\widehat{y_i} - y_i)^2 + \lambda \sum_{j=1}^{n} |\theta_j|$$

the first term is the least squares loss, representing the squared difference between predicted and actual values.

- the second term is the L1 regularization term, it penalizes the sum of absolute values of the regression coefficient θj.

2. Ridge Regression (L2 Regularization)

Ridge regression is a linear regression technique that adds a regularization term to the standard linear objective. Again, the goal is to prevent overfitting by penalizing large coefficient in linear regression equation. It useful when the dataset has multicollinearity where predictor variables are highly correlated.

The objective function after applying ridge regression is:

$$J(\theta) = \frac{1}{2m}\sum_{i=1}^{m}(\widehat{y_i} - y_i)^2 + \lambda\sum_{j=1}^{n}\theta_j^2$$

- the first term is the least squares loss, representing the squared difference between predicted and actual values.
- the second term is the L1 regularization term, it penalizes the sum of square of values of the regression coefficient θj.

3. Elastic Net Regression

Elastic Net Regression is a hybrid regularization technique that combines the power of both L1 and L2 regularization in linear regression objective.

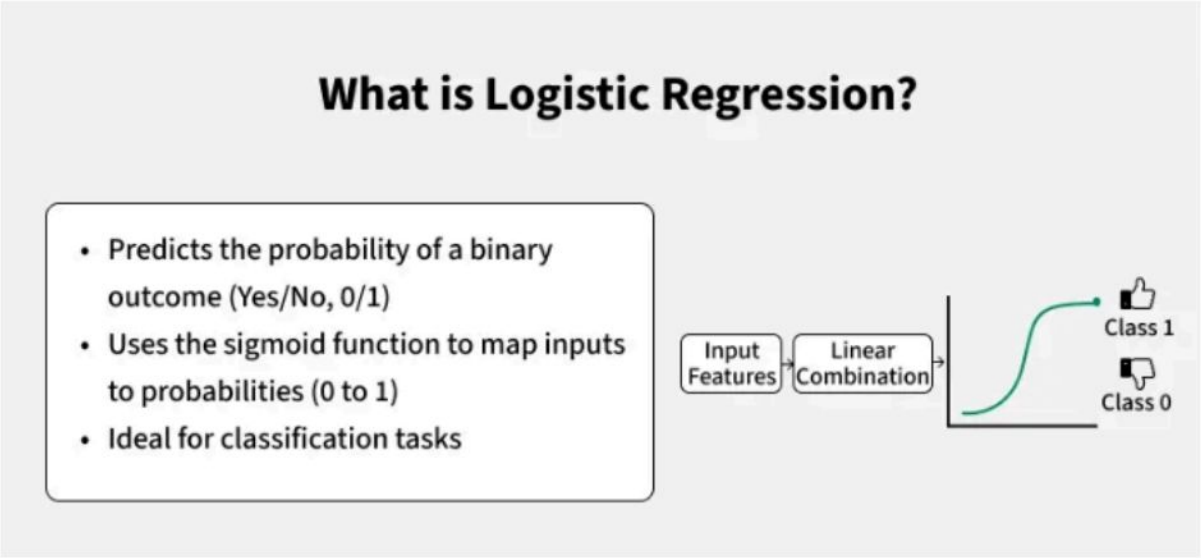
$$J(\theta) = \frac{1}{2m}\sum_{i=1}^{m}(\widehat{y_i} - y_i)^2 + \alpha\lambda\sum_{j=1}^{n}|\theta_j| + \frac{1}{2}(1-\alpha)\lambda\sum_{j=1}n\theta_j^2$$

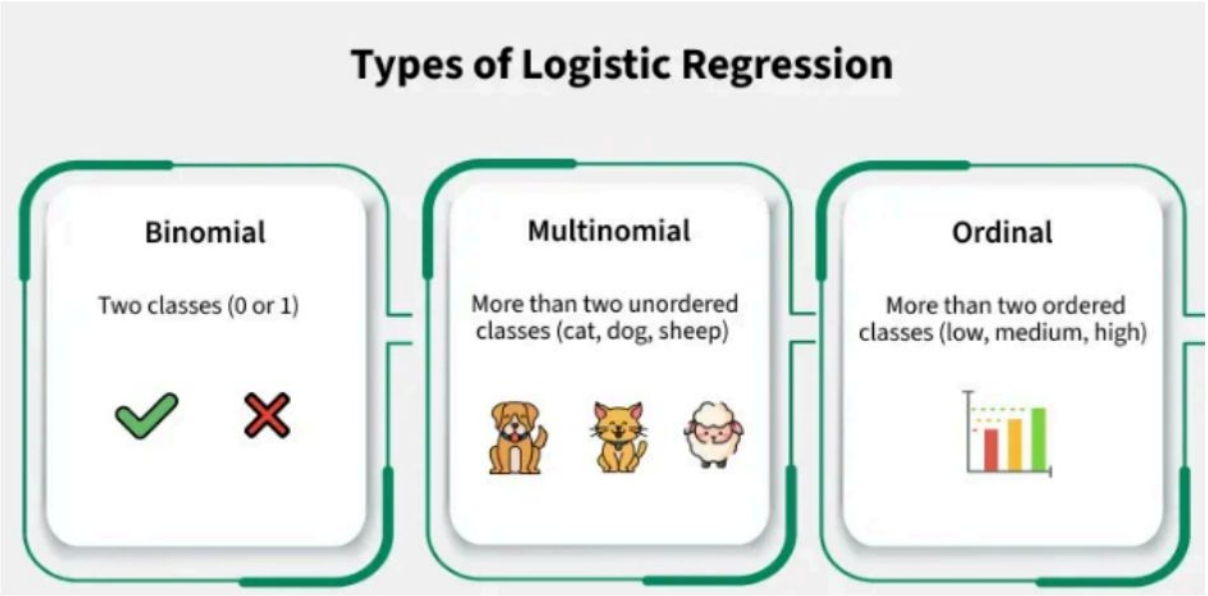
- the first term is least square loss.
- the second term is L1 regularization and third is ridge regression.
- Λ is the overall regularization strength.
- A controls the mix between L1 and L2 regularization.

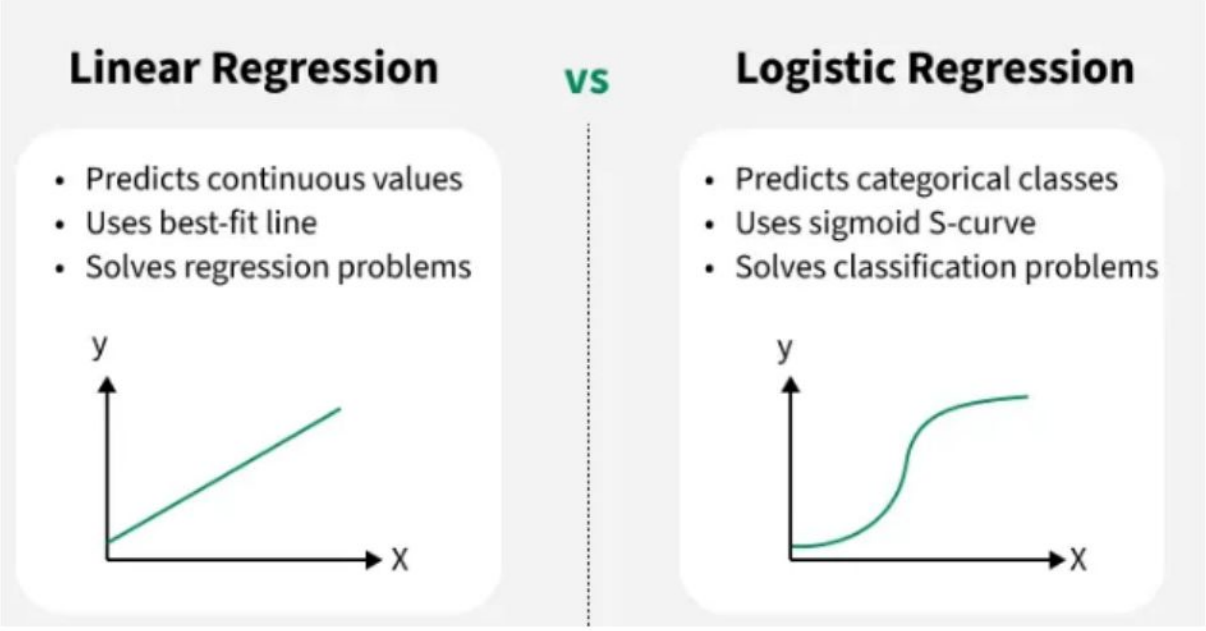
Now that we have learned how to make a linear regression model, now we will implement it.

Logistic Regression in Machine Learning

Logistic Regression is a **supervised machine learning algorithm used for classification problems.** Unlike linear regression which predicts continuous values it predicts the probability that an input belongs to a specific class. It is used for binary classification where the output can be one of two possible categories such as Yes/No, True/False or 0/1. It uses **sigmoid function to convert inputs into a probability value between 0 and 1.** In this article, we will see the basics of logistic regression and its core concepts.







Types of Logistic Regression

Logistic regression can be classified into three main types based on the nature of the dependent variable:

1. **Binomial Logistic Regression**: This type is used when the dependent variable has only two possible categories. Examples include Yes/No, Pass/Fail or 0/1. It is the most common form of logistic regression and is used for binary classification problems.
2. **Multinomial Logistic Regression**: This is used when the dependent variable has three or more possible categories that are not ordered. For example, classifying animals into categories like "cat," "dog" or "sheep." It extends the binary logistic regression to handle multiple classes.
3. **Ordinal Logistic Regression**: This type applies when the dependent variable has three or more categories with a natural order or ranking. Examples include ratings like "low," "medium" and "high." It takes the order of the categories into account when modeling.

Assumptions of Logistic Regression

Understanding the assumptions behind logistic regression is important to ensure the model is applied correctly, main assumptions are:

1. **Independent observations**: Each data point is assumed to be independent of the others means there should be no correlation or dependence between the input samples.
2. **Binary dependent variables**: It takes the assumption that the dependent variable must be binary, means it can take only two values. For more than two categories SoftMax functions are used.
3. **Linearity relationship between independent variables and log odds**: The model assumes a linear relationship between the independent variables and the log odds of the dependent variable which means the predictors affect the log odds in a linear way.
4. **No outliers**: The dataset should not contain extreme outliers as they can distort the estimation of the logistic regression coefficients.
5. **Large sample size**: It requires a sufficiently large sample size to produce reliable and stable results.

Understanding Sigmoid Function

1. The sigmoid function is a important part of logistic regression which is used to convert the raw output of the model into a probability value between 0 and 1.

2. This function takes any real number and maps it into the range 0 to 1 forming an "S" shaped curve called the sigmoid curve or logistic curve. Because probabilities must lie between 0 and 1, the sigmoid function is perfect for this purpose.

3. In logistic regression, we use a threshold value usually 0.5 to decide the class label.

- If the sigmoid output is same or above the threshold, the input is classified as Class 1.

- If it is below the threshold, the input is classified as Class 0.

This approach helps to transform continuous input values into meaningful class predictions.

How does Logistic Regression work?

Logistic regression model transforms the linear regression function continuous value output into categorical value output using a sigmoid function which maps any real-valued set of independent variables input into a value between 0 and 1. This function is known as the logistic function.

$$X = \begin{bmatrix} x_{11} & \cdots & x_{1m} \\ x_{21} & \cdots & x_{2m} \\ \vdots & \ddots & \vdots \\ x_{n1} & \cdots & x_{nm} \end{bmatrix}$$

and the dependent variable is Y having only binary value i.e 0 or 1.

$$Y = \begin{cases} 0 & \text{if } Class\ 1 \\ 1 & \text{if } Class\ 2 \end{cases}$$

then, apply the multi-linear function to the input variables X.

$$z = \left(\sum_{i=1}^{n} w_i x_i\right) + b$$

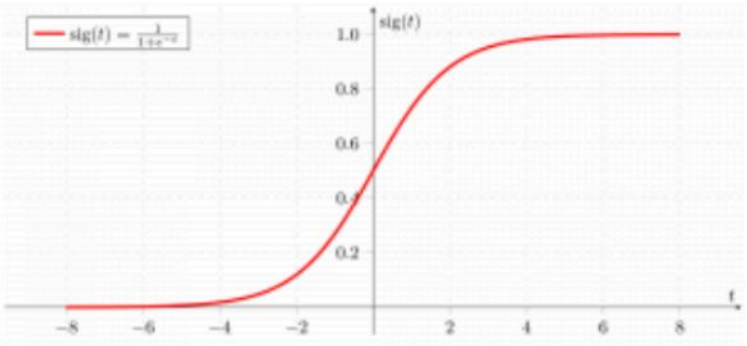
Here x_i is the ith observation of X, $w_i = [w_1, w_2, w_3, \cdots, w_m]$ is the weights or Coefficient and $b$is the bias term also known as intercept. Simply this can be represented as the dot product of weight and bias.

$$z = w \cdot X + b$$

At this stage, $z$is a continuous value from the linear regression. Logistic regression then applies the sigmoid function to $z$to convert it into a probability between 0 and 1 which can be used to predict the class.

Now we use the sigmoid function where the input will be z and we find the probability between 0 and 1. i.e. predicted y.

$$\sigma(z) = \frac{1}{1+e^{-z}}$$



Sigmoid function

As shown above the sigmoid function converts the continuous variable data into the probability i.e between 0 and 1.

- $\sigma(z)$ tends towards 1 as $z \to \infty$
- $\sigma(z)$ tends towards 0 as $z \to -\infty$
- $\sigma(z)$ is always bounded between 0 and 1

where the probability of being a class can be measured as:

$$P(y = 1) = \sigma(z)$$
$$P(y = 0) = 1 - \sigma(z)$$

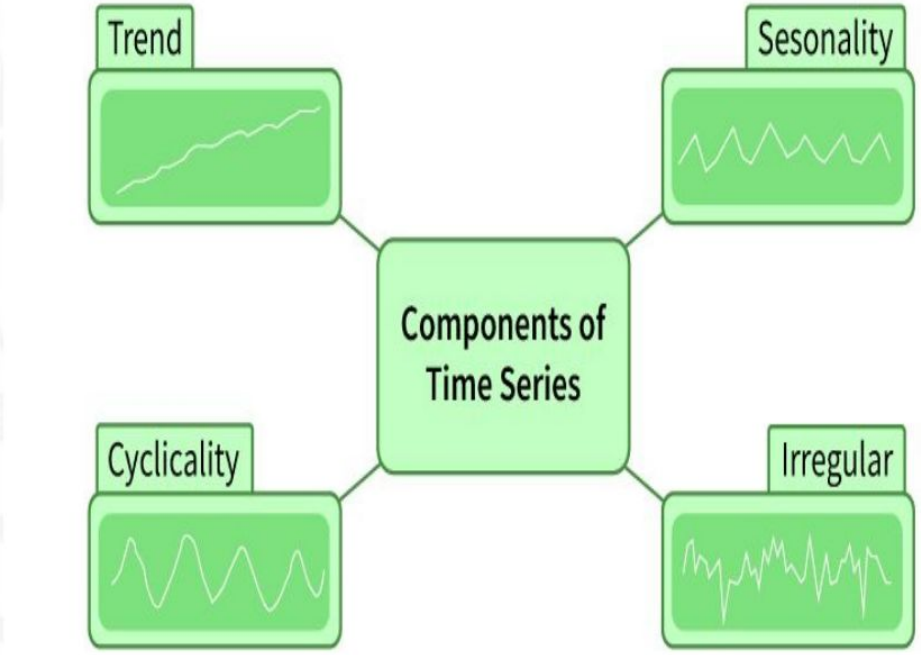
Terminologies involved in Logistic Regression

1. **Independent Variables:** These are the input features or predictor variables used to make predictions about the dependent variable.
2. **Dependent Variable**: This is the target variable that we aim to predict. In logistic regression, the dependent variable is categorical.

3. **Logistic Function**: This function transforms the independent variables into a probability between 0 and 1 which represents the likelihood that the dependent variable is either 0 or 1.
4. **Odds**: This is the ratio of the probability of an event happening to the probability of it not happening. It differs from probability because probability is the ratio of occurrences to total possibilities.
5. **Log-Odds (Logit)**: The natural logarithm of the odds. In logistic regression, the log-odds are modeled as a linear combination of the independent variables and the intercept.
6. **Coefficient**: These are the parameters estimated by the logistic regression model which shows how strongly the independent variables affect the dependent variable.
7. **Intercept**: The constant term in the logistic regression model which represents the log-odds when all independent variables are equal to zero.
8. **Maximum Likelihood Estimation (MLE)**: This method is used to estimate the coefficients of the logistic regression model by maximizing the likelihood of observing the given data.

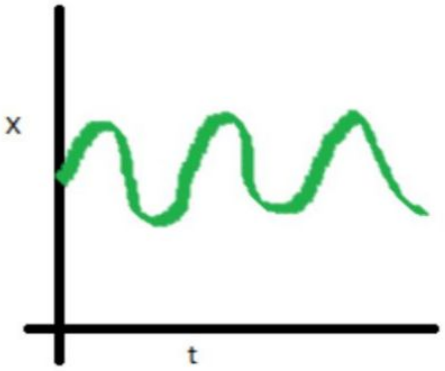
Components of Time Series

1. **DataTrend:** Trend represents the long-term movement or directionality of the data over time. It captures the overall tendency of the series to increase, decrease, or remain stable.
2. **Seasonality:** Seasonality refers to periodic fluctuations or patterns that occur at regular intervals within the time series..
3. **Cyclic variations:** Cyclical variations are longer-term fluctuations in the time series that do not have a fixed period like seasonality.
4. **Irregularity (or Noise):** Irregularity, also known as noise or randomness, refers to the unpredictable or random fluctuations in the data that cannot be attributed to the trend, seasonality, or cyclical variations.

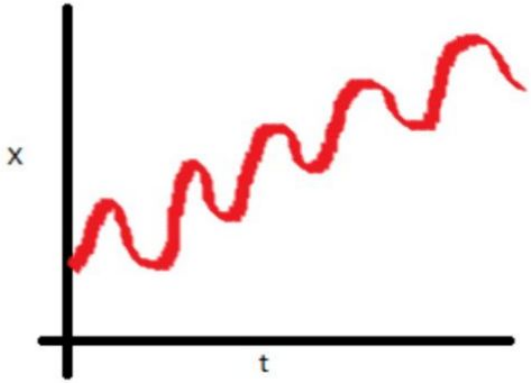


Time Series - Terms

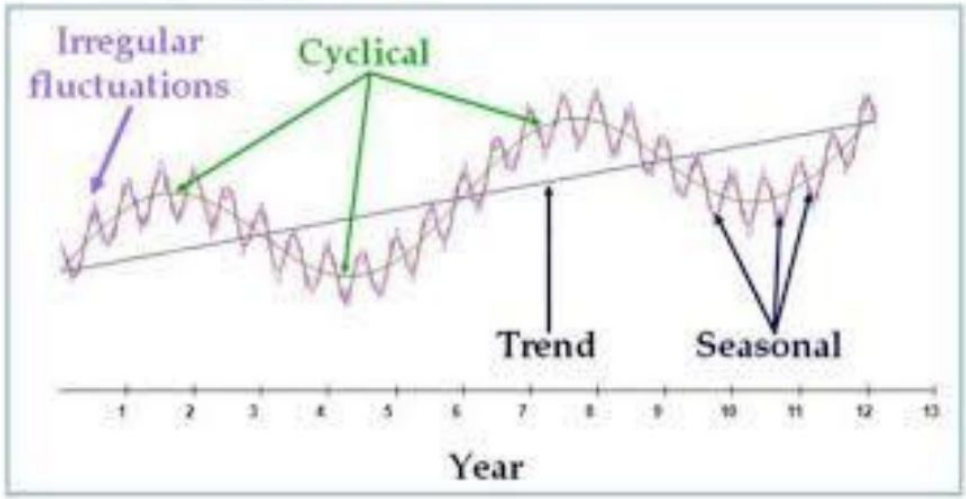
- Stationary Data - a time series variable exhibiting no significant upward or downward trend over time.
- Nonstationary Data - a time series variable exhibiting a significant upward or downward trend over time.
- Seasonal Data - a time series variable exhibiting a repeating patterns at regular intervals over time.



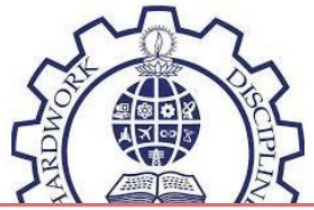
Stationary series



Non-Stationary series



Preprocessing of Time Series Data



DIFFERENCING	TRANSFORMATION	DECOMPOSITION	DETRENDING	DESPIKING
A technique to remove series dependencies, often used to stabilize the mean. Typically applied to reduce trends and seasonality for stationarity.	A method to stabilize varying amplitude, commonly used in series with exponential growth or decay to ensure constant variance.	The process of breaking a series into trend, seasonality, and residuals. This can be used to understand the underlying patterns in the data and to manage each component separately.	A strategy to remove underlying series trends. Used to focus on cycles or irregularities by isolating them from long-term trends.	The act of smoothing out abrupt spikes. Applied to minimize the influence of extreme values or outliers on models and insights.

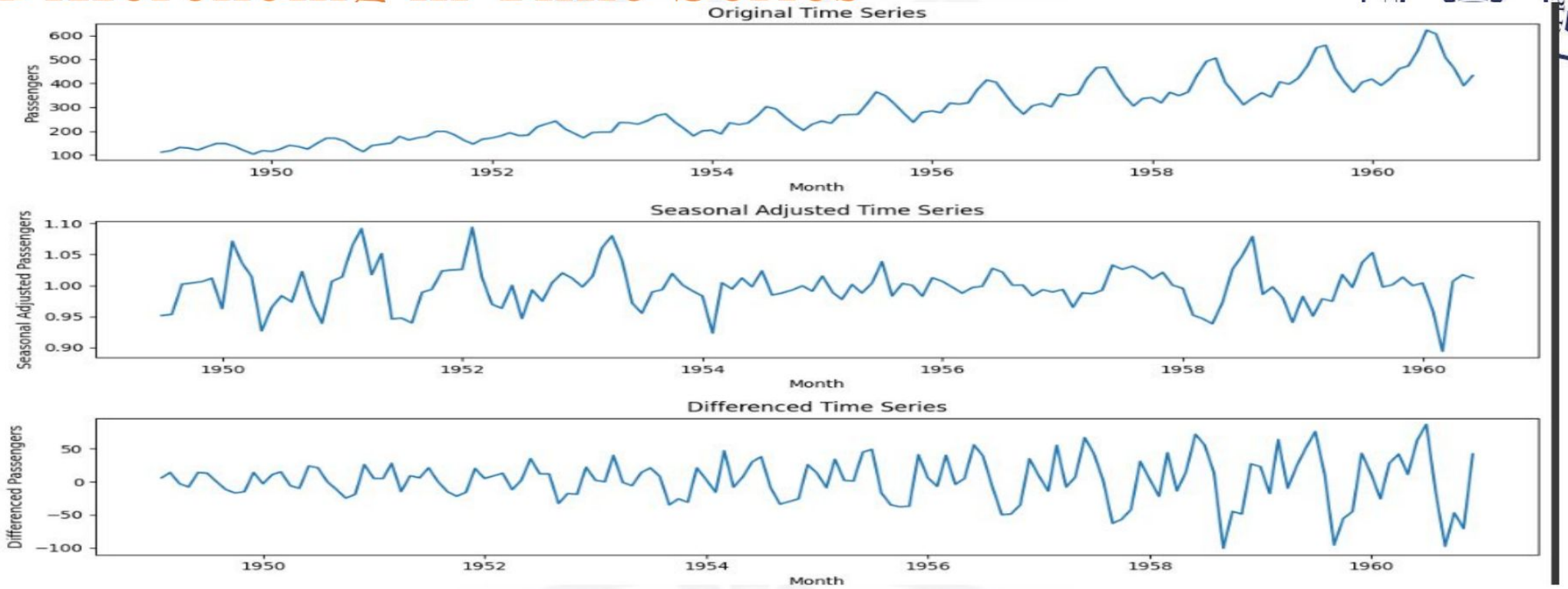
- Time series preprocessing refers to the steps taken to clean, transform, and prepare time series data for analysis or forecasting.
- The goal is to ensure that the time series data is in a suitable format for subsequent analysis or modeling.
  - **Handling Missing Values :** Dealing with missing values in the time series data to ensure continuity and reliability in analysis.
  - **Dealing with Outliers:** Identifying and addressing observations that significantly deviate from the rest of the data, which can distort analysis results.
  - **Stationarity and Transformation:** Ensuring that the statistical properties of the time series, such as mean and variance, remain constant over time. Techniques like differencing, detrending, and deseasonalizing are used to achieve stationarity.

Differencing in Time Series



- The process of calculating the differences between successive observations in a given time series is known as differencing.
- The main goal of differencing is to remove non-constant variation and trends from a time series while also stabilizing the mean. When working with non-stationary data—where the statistical characteristics of the series fluctuate over time.
- Differences are useful when dealing with nonstationary time series data, where the mean, variance, or other statistical features change over time The observation and analysis of nonstationary data can be more complicated, and differentiation is a common method of converting such information into a stable form
- In time series analysis, seasonal adjustment and differencing are two crucial methods. A time series' trend component may be eliminated using differencing, but the seasonal component can be eliminated using seasonal adjustment. Forecasting models may be made more accurate by using both strategies.

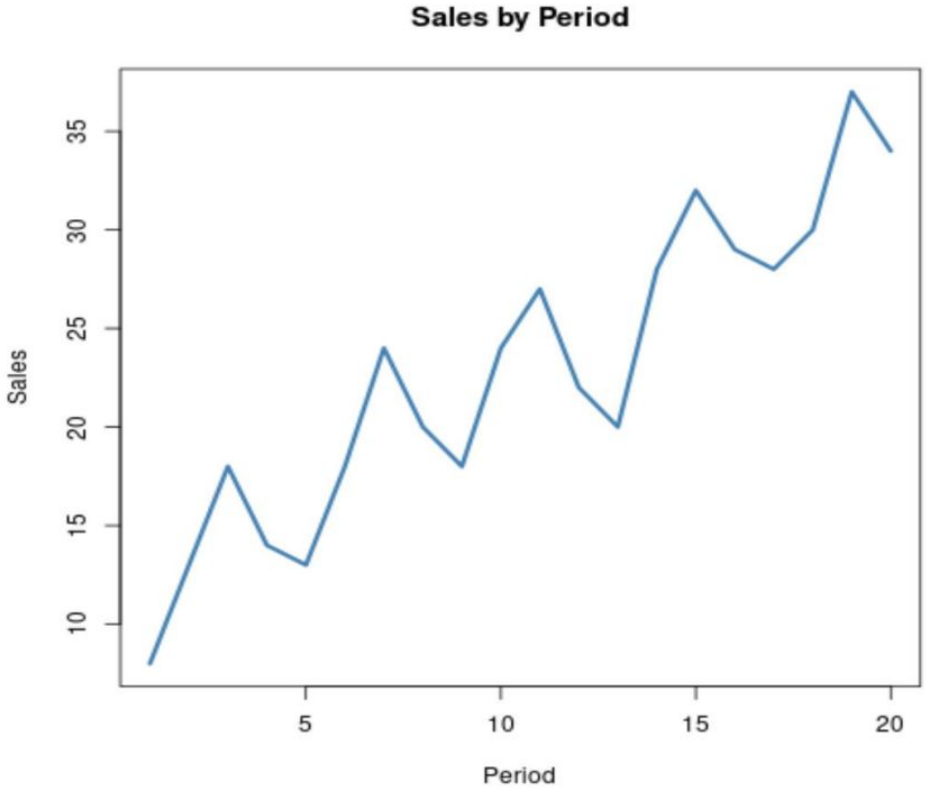
Differencing in Time Series



- The original time series is shown in the first subplot.
- The adjusted for seasons time series is shown in the second subplot.
- The distinct time series data are shown in the third subplot.

Detrending in Time Series

To “detrend” time series data means to remove an underlying trend in the data. The main reason we would want to do this is to more easily see subtrends in the data that are seasonal or cyclical.



For example, consider the following time series data that represents the total sales for some company during 20 consecutive periods: Clearly the sales is trending upwards over time, but there also appears to be a cyclical or seasonal trend in the data, which can be seen by the tiny “hills” that occur over time.

Detrending in Time Series

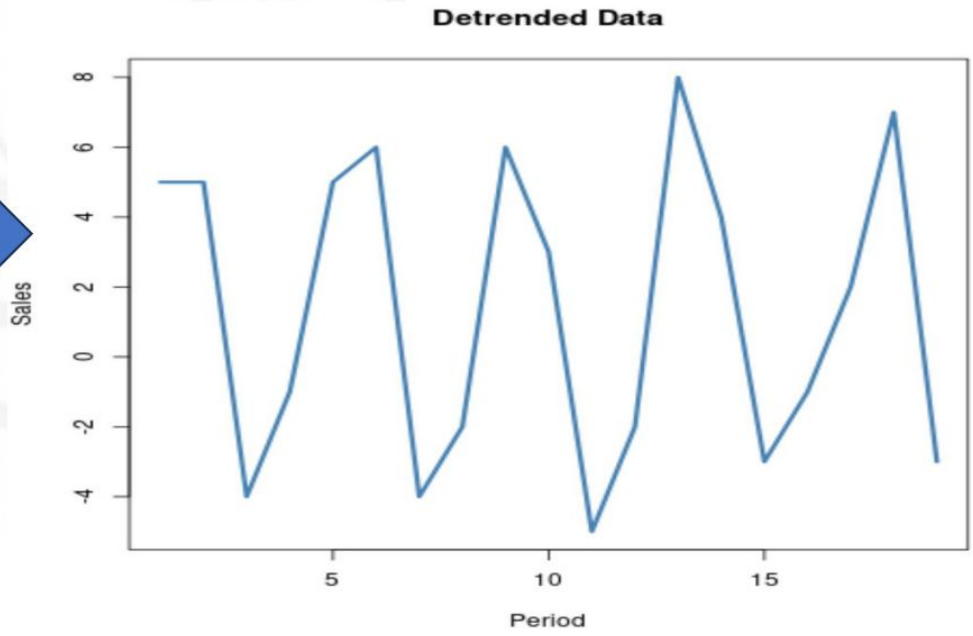
Original Data	Detrended Data
8	
13	5
18	5
14	-4
13	-1
18	5
24	6
20	-4
18	-2
24	6
27	3
22	-5
20	-2
28	8
32	4
29	-3
28	-1
30	2
37	7
34	-3

There are two common methods used to detrend time series data:

1. Detrend by Differencing

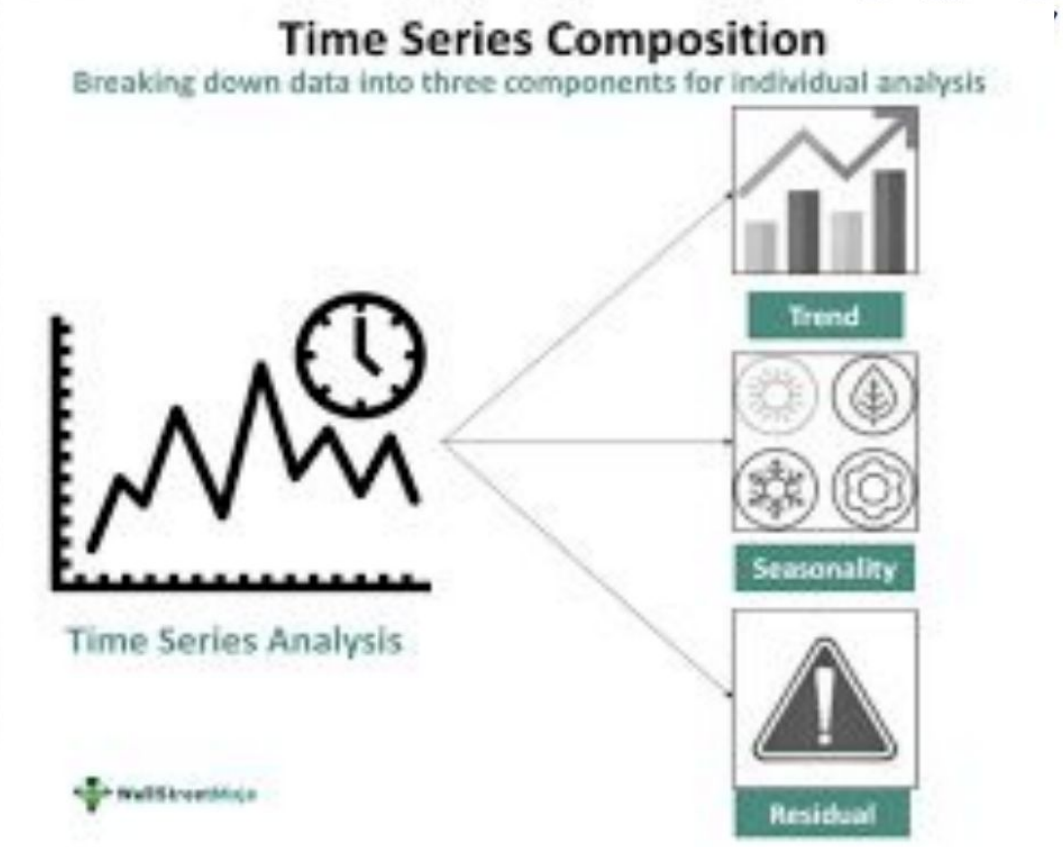
2. Detrend by Model Fitting

To obtain the first value of the detrended time series data, we calculate 13 – 8 = 5. Then to obtain the next value we calculate 18-13 = 5, and so on.



Time Series Analysis and Decompositon

- Time Series Analysis and Decomposition is a systematic approach to studying sequential data collected over successive time intervals.
- It involves analyzing the data to understand its underlying patterns, trends, and seasonal variations, as well as decomposing the time series into its fundamental components.
- This decomposition typically includes identifying and isolating elements such as trend, seasonality, and residual (error) components within the data.



Different Time Series Analysis Techniques



Box-Jenkins Method: Box-Jenkins Method is a systematic approach for analyzing and modeling time series data. It involves identifying the appropriate autoregressive integrated moving average (ARIMA) model parameters, estimating the model, diagnosing its adequacy through residual analysis, and selecting the best-fitting model.

Autocorrelation Analysis: A statistical method to measure the correlation between a time series and a lagged version of itself at different time lags. It helps identify patterns and dependencies within the time series data.

Partial Autocorrelation Functions (PACF): PACF measures the correlation between a time series and its lagged values, controlling for intermediate lags, aiding in identifying direct relationships between variables.

Different Time Series Analysis Techniques



In time series analysis, the moving-average model (MA model), also known as moving-average process, is a common approach for modeling univariate time series. The moving-average model specifies that the output variable is cross-correlated with a non-identical to itself random-variable.

Moving Average Technique

Monthly sales revenue data were collected for a company for 20X2:

	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
Sales $000	125	145	186	131	151	192	137	157	198	143	163	204

- A moving average is a series of averages, calculated from historic data.
- Moving averages can be calculated for any number of time periods, for example a three-month moving average, a seven-day moving average, or a four-quarter moving average.

Moving Average Technique

Month	Sales ($000)	Three-month moving total ($000)	Three-month moving average ($000)	Seasonal variation ($000)
January	125			
February	145	456	152	(145 – 152) = -7
March	186	462	154	(186 – 154) = 32
April	131	468	156	(131 – 156) = -25
May	151	474	158	(151 – 158) = -7
June	192	480	160	(192 – 160) = 32
July	137	486	162	(137 – 162) = -25
August	157	492	164	(157 – 164) = -7
September	198	498	166	(198 – 166) = 32
October	143	504	168	(143 – 168) = -25
November	163	510	170	(163 – 170) = -7
December	204			

- A negative variation means that the actual figure in that period is less than the trend and a positive figure means that the actual is more than the trend.
- From the data we can see a clear three-month cycle in the seasonal variation. Every first month has a variation of -7, suggesting that this month is usually $7,000 below the average.

Every second month has a variation of 32 suggesting that this month is usually $32,000 above the average. In month 3, the variation suggests that every third month, the actual will be $25,000 below the average

Time Series Analysis

- A *time-series* is a set of observations on a quantitative variable collected over time.
- Examples
  - Dow Jones Industrial Averages
  - Historical data on sales, inventory, customer counts, interest rates, costs, etc
- Businesses are often very interested in forecasting time series variables.
- Often, independent variables are not available to build a regression model of a time series variable.



In time series analysis, we analyze the past behavior of a variable in order to predict its future behavior.